Lecture 02
Topics: **Data representation, Boolean expression**

CIS-310: Computer Organization & Assembly Language

MW

Probir Roy
Assistant professor in CIS

DEARBORN

Topics

- Data representation
  - Binary representation
  - Hexa-decimal representation
  - Signed number representation
  - Translation between representations
  - Addition and Subtraction operation in binary and hexa-decimal representation
- Boolean expression
  - Boolean operators
  - Truth-table for Boolean expression

- Reading:
  - https://faculty.etsu.edu/tarnoff/ntes2150/Ch3_v02.pdf (chapter 3)
  - https://faculty.etsu.edu/tarnoff/ntes2150/Ch4_v02.pdf (chapter 4.1, 4.2)

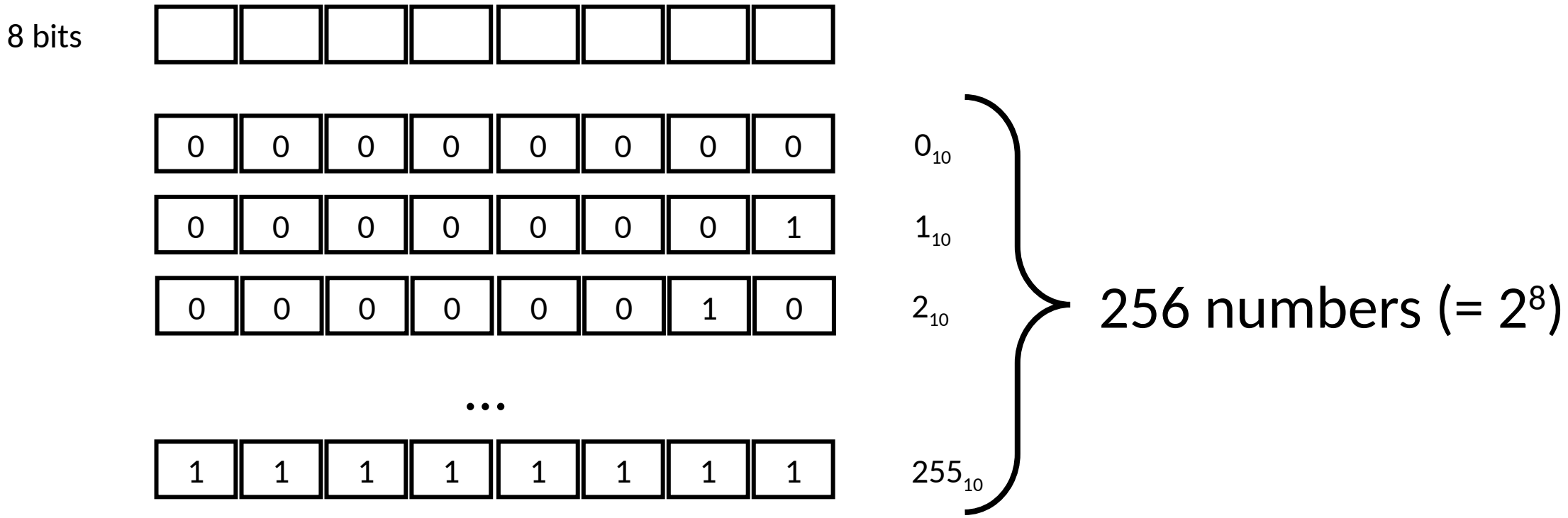
Data representation

- Binary numbers: Base 2 (Example: 1100, 0101)
- Decimal numbers: Base 10 (Example: 839, 510)
- Hexadecimal numbers: Base 16 (Example: 5AE, 10C)
  - We often use hexadecimal numbers to quickly understand binary data.

Binary integers

- Represented with sequence of **Bits**
- We can represent a number with 8 bit, 16 bit, 32 bit, 64 bit …
- 8 bit representation of decimal 15 = 0000 1111
- 16 bit representation of decimal 15 = 0000 0000 0000 1111
- 32 bit representation of decimal 15 = 0000 0000 0000 0000 0000 0000 0000 1111

How many numbers we can represent with 8 bit?



Minimum number 0 and Maximum number is 255 (= 2^8 - 1)

Signed vs unsigned integer

$3_{10} = 0011_2$

$-3_{10} = 1101_2$

$4_{10} = 0100_2$

$-4_{10} = 1100_2$

Deriving Two’s complement representation

- For instance, we want to know binary represntaion -5 or two’s complement of 5
  - Step 1) Unsigned representation of the number 5= 0000 0101
  - Step 2) Reverse the bits: 1111 1010
  - Step 3) Add +1 to step 2: 1111 1011

Deriving Two’s complement representation

10 – 5 = 10 + (-5)

0000 1010

1111 1011 (+)

0000 0101

Carry = 1

Example (-7)

Step 1 : binary representation of 7 = 0000 0111

Step 2: (Reverse the bit values) : 1111 1000

Step 3: (Add +1 to step 2) : 1111 1001

-7 = 1111 1001

+7 = 0000 0111

(C=1) 0000 0000

Two’s complement of –N is N

- Two’s complement of (-7)

Step 1) (-7 binary representation) = 1111 1001

Step 2) (Reverse the bit values) = 0000 0110

Step 3) Add +1 to step 2 = 0000 0111 = 7

Two’s complement on hexadecimal numbers

What’s the Two’s complement of 1A0F ?

Step 1: 1 A 0 F

Step 2: Reverse the bit values E 5 F 0

(Reverse in Hex = 0 -> F, 1 -> E, 2 -> D …. E-> 1, F -> 0)

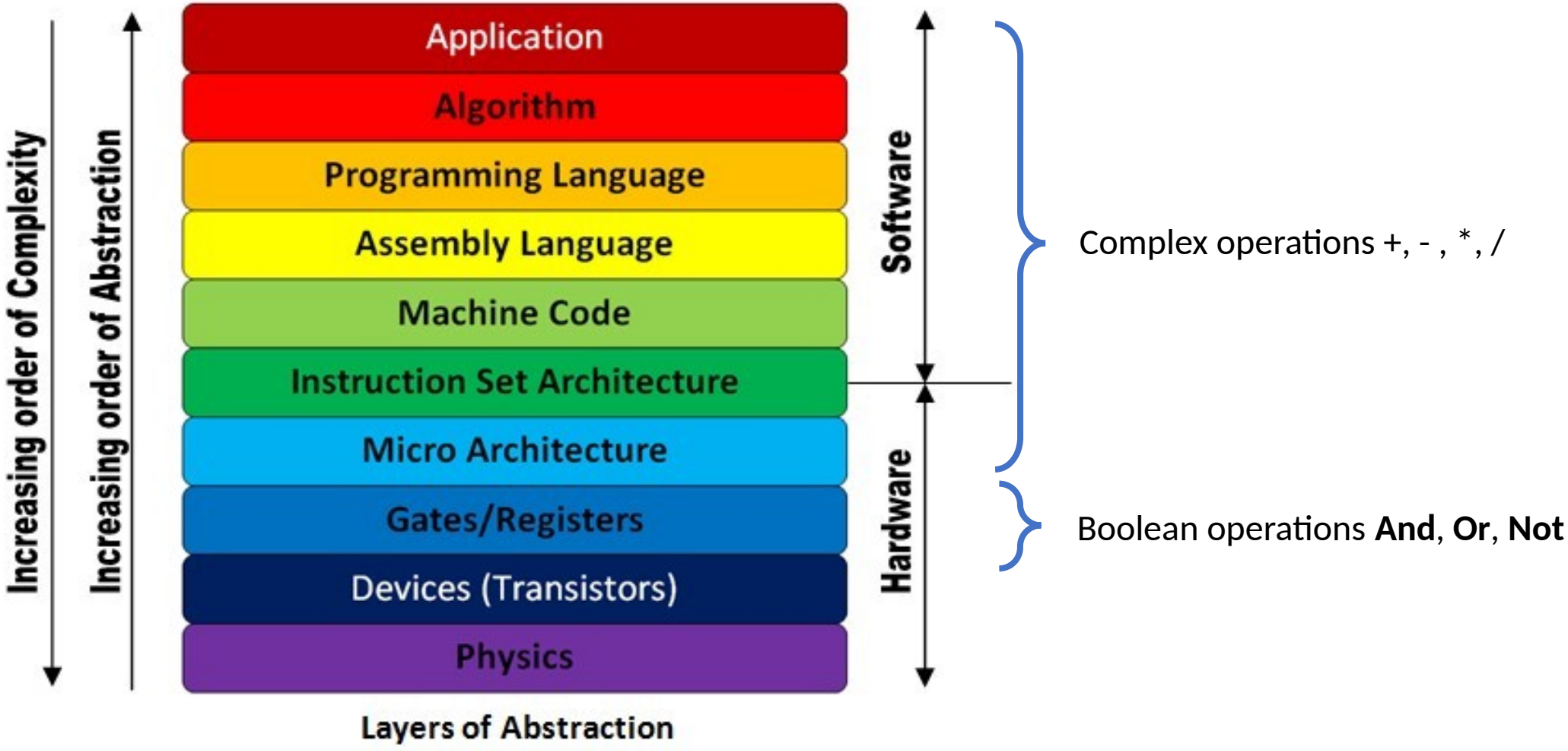
Step 3: +1 E 5 F 1

Practice

- What is 8-bit signed binary representation of decimal (-31)?

Boolean Expression

Operation


Application
Algorithm
Programming Language
Assembly Language
Machine Code
Instruction Set Architecture
Micro Architecture
Gates/Registers
Devices (Transistors)
Physics
Increasing order of Complexity
Increasing order of Abstraction
Software
Hardware
Layers of Abstraction
Complex operations +, - , *, /
Boolean operations **And**, **Or**, **Not**

Arithmetic operations and Boolean algebra

- Arithmetic operations are function of Boolean algebra
  - For example:
  - Addition = f(Boolean operations) = f(And, Or, Not)

- We need to understand the basics of Boolean algebra

Expressions in Boolean algebra

Expression	Description
$\neg X$	NOT X
$X \wedge Y$	X AND Y
$X \vee Y$	X OR Y
$\neg X \vee Y$	(NOT X) OR Y
$\neg (X \wedge Y)$	NOT (X AND Y)
$X \wedge \neg Y$	X AND (NOT Y)

Boolean Operations (NOT, AND, OR)

X	(NOT X) ~X
True	False
False	True

X	Y	X AND Y
True	True	True
False	True	False
True	False	False
False	False	False

X	Y	X OR Y
True	True	True
False	True	**True**
True	False	**True**
False	False	False

Boolean Operations (NOT, AND, OR)

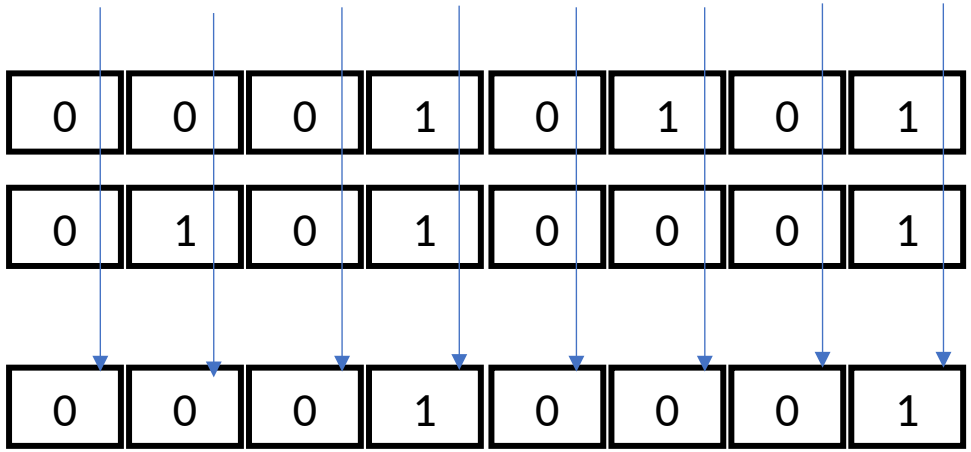
X	(NOT X) ~X
1	0
0	1

X	Y	X AND Y
1	1	1
0	1	0
1	0	0
0	0	0

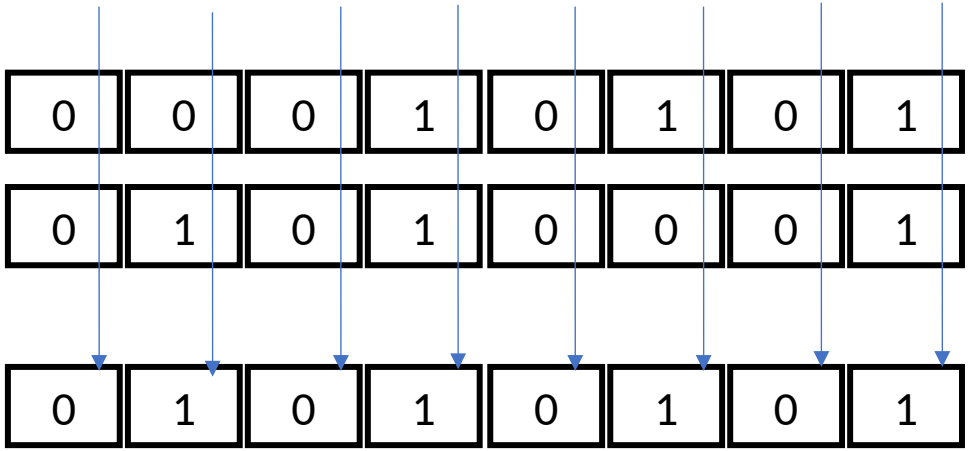
X	Y	X OR Y
1	1	1
0	1	**1**
1	0	**1**
0	0	0

Boolean operation 8 bit binary

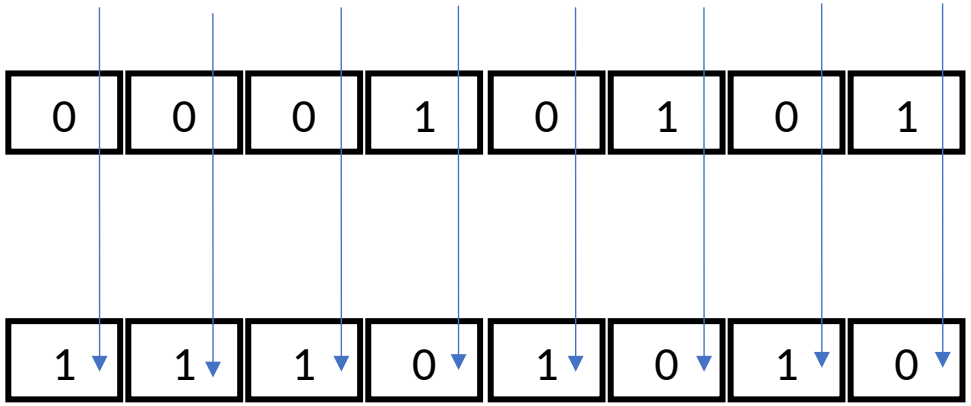
Boolean AND operation



Boolean OR operation



Boolean NOT operation



Boolean expression

Expression	Order of Operations
$\neg X \vee Y$	NOT, then OR
$\neg(X \vee Y)$	OR, then NOT
$X \vee (Y \wedge Z)$	AND, then OR

Truth table of Boolean functions

Example: $\neg X \vee Y$

X	$\neg X$	Y	$\neg X \vee Y$
F	T	F	T
F	T	T	T
T	F	F	F
T	F	T	T

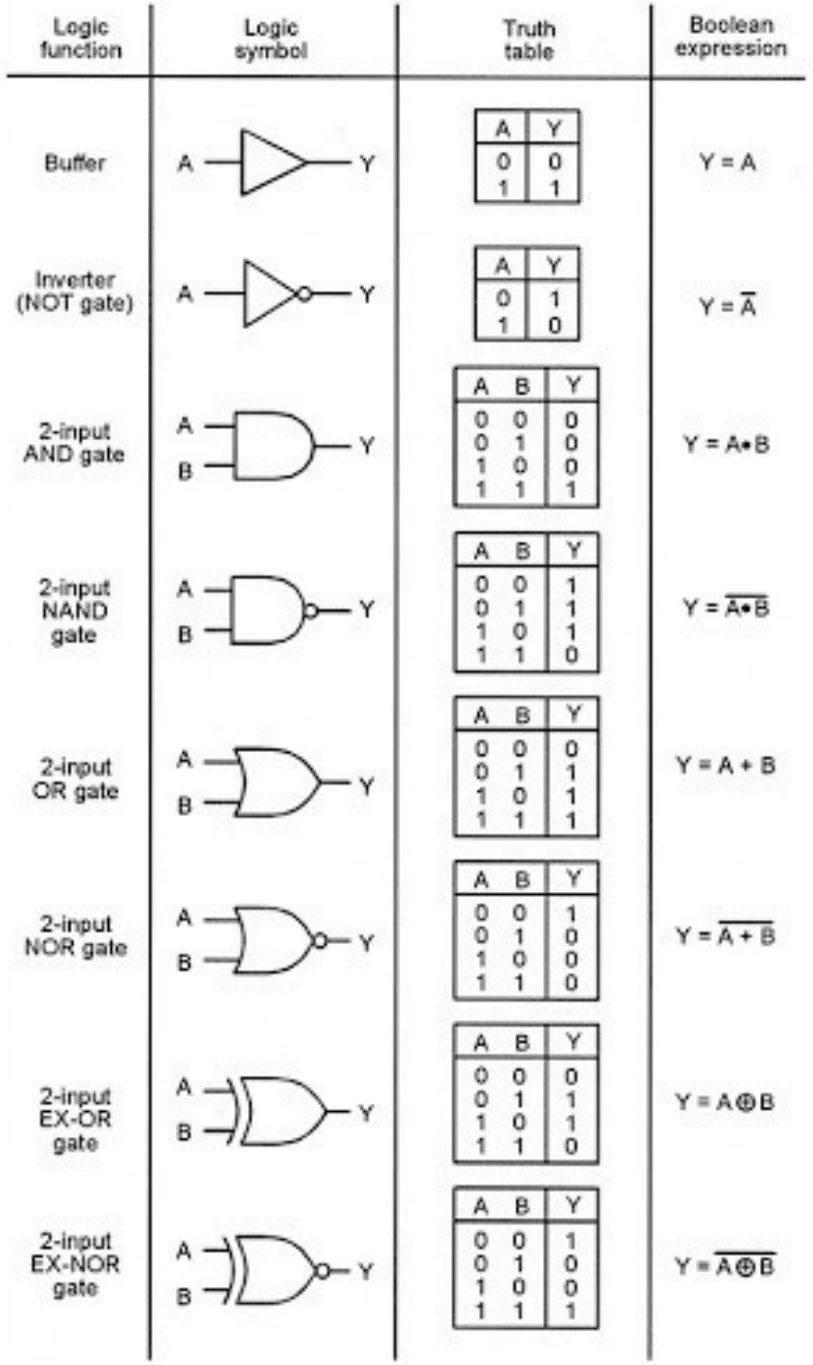
Truth Table (Example)

- Example: $(Y \wedge S) \vee (X \wedge \neg S)$

X	Y	S	$Y \wedge S$	$\neg S$	$X \wedge \neg S$	$(Y \wedge S) \vee (X \wedge \neg S)$
F	F	F	F	T	F	F
F	T	F	F	T	F	F
T	F	F	F	T	T	T
T	T	F	F	T	T	T
F	F	T	F	F	F	F
F	T	T	T	F	F	T
T	F	T	F	F	F	F
T	T	T	T	F	F	T

Truth Table of 1 bit Adder

Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



A Logic gate is a hardware implementation of Boolean operation

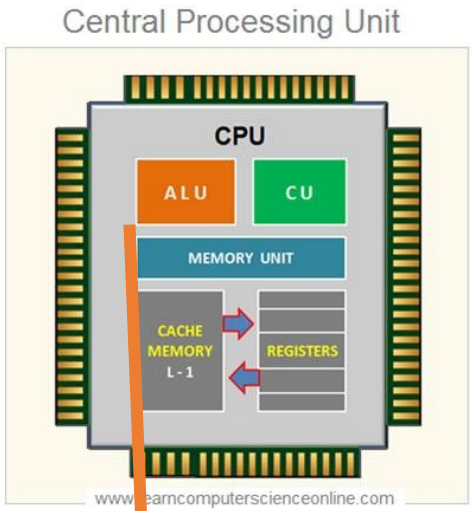
Try on a simulator

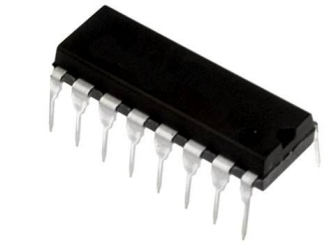
https://circuitverse.org/users/11440/projects/cis310

Implementing a Boolean expression with logic gates

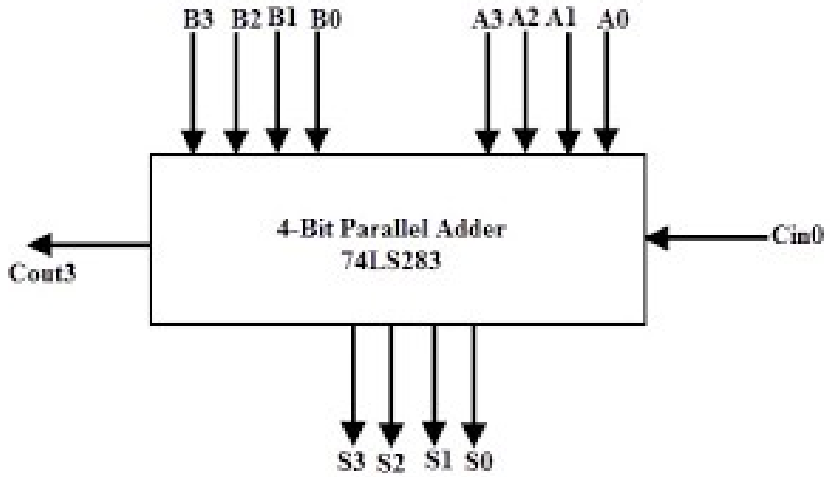
- Processor functionalities are implemented using logic gates.
- At first we express the processor functionality with Boolean expression.
- We then construct logic circuit of the expression using logic gates

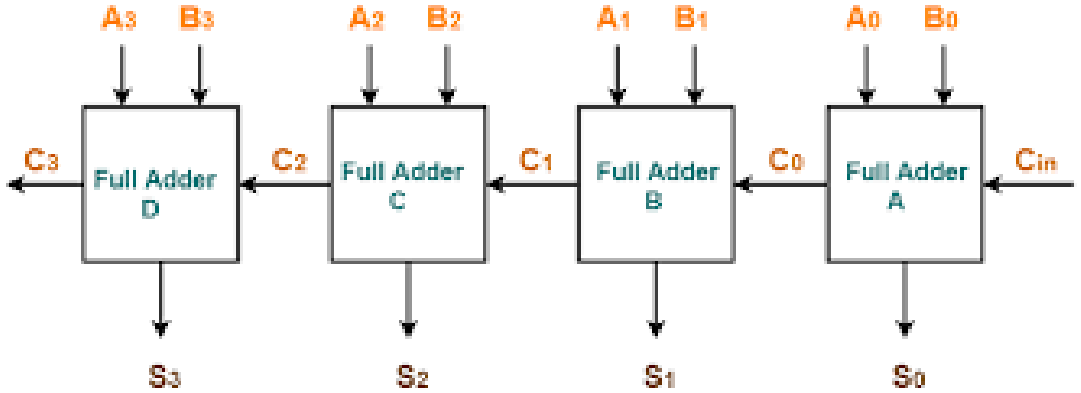
Adder circuit in ALU





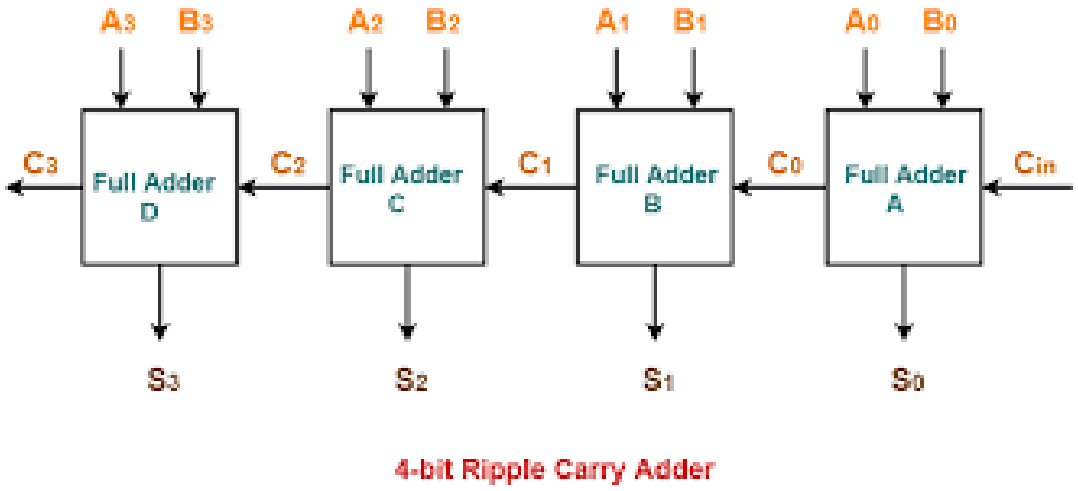
Adder





4-bit Ripple Carry Adder

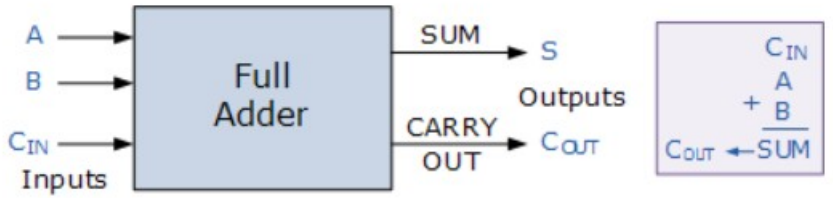
4 1-bit full adder



4-bit Ripple Carry Adder

C3	C2	C1	C0	Cin	Carry
	A3	A2	A1	A0	
+	B3	B2	B1	B0	
	S3	S2	S1	S0	Sum

Full adder in Truth table

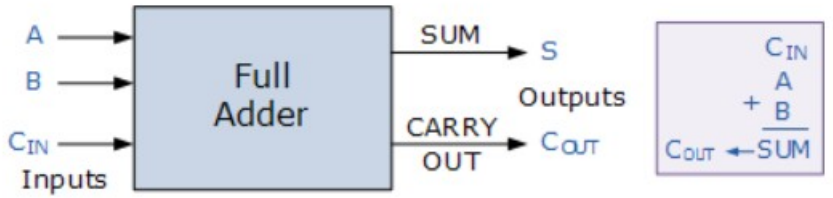


Sum(A,B, Cin) = ?

Carry(A,B, Cin) = ?

Truth Table				
C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full adder in Boolean expression

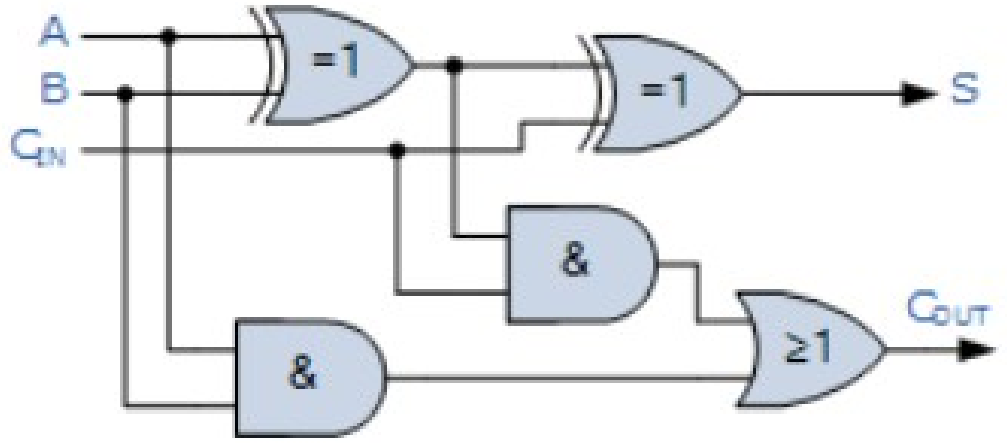


Truth Table				
C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

SUM = (A XOR B) XOR Cin = (A ⊕ B) ⊕ Cin

CARRY-OUT = A AND B OR Cin(A XOR B) = A.B + Cin(A ⊕ B)

Implementing a full adder with logic gates



SUM = (A XOR B) XOR Cin = (A ⊕ B) ⊕ Cin

CARRY-OUT = A AND B OR Cin(A XOR B) = A.B + Cin(A ⊕ B)

Try it https://circuitverse.org/users/11440/projects/cis310

Topics we covered today

- Data representation
  - Binary representation
  - Hexa-decimal representation
  - Signed number representation
  - Translation between representations
  - Addition and Subtraction operation in binary and hexa-decimal representation
- Boolean expression
  - Boolean operators
  - Truth-table for Boolean expression